EXPERIMENT 13

Case Study on the Limitations of Low-Code Platforms for a Complex Enterprise Use Case

Global Loan Processing Enterprise (Platform-Neutral Case Study)

Submitted by

Mohid Ahmed F

Register Number: **241401057**

1. Aim

To examine, through a case study, the limits that low-code platforms as a category reach when they are applied to a complex enterprise system, rather than judging any single vendor. The scenario is a bank that handles loan applications in many countries — an enterprise-scale extension of the Bank Loan Application Form built in the earlier experiments — and the study ends with suggestions for keeping those limits under control.

The case study asks three things: where low-code clearly helps, where it begins to struggle as scale and complexity increase, and how a hybrid design can keep the speed of low-code while covering its weak points.

2. Procedure

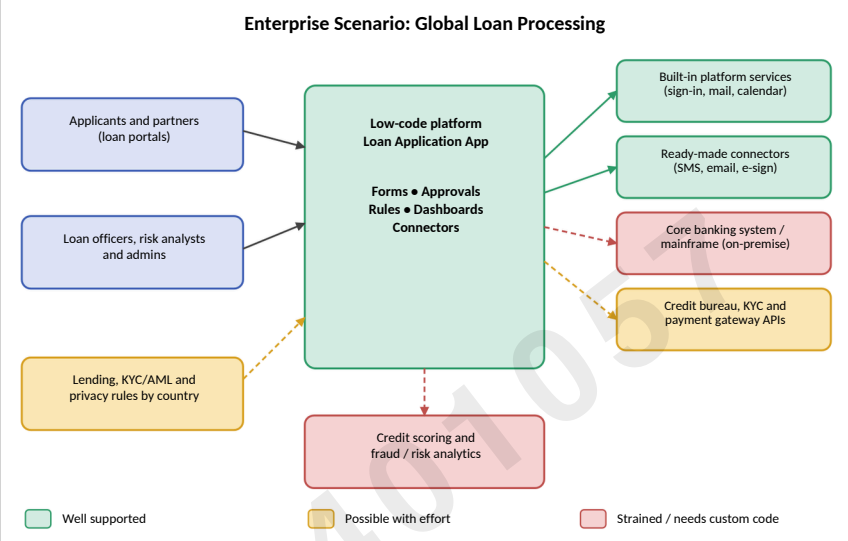
24140 1057

1. **Set the scenario.** A global bank was described that takes loan applications through web and mobile portals, serves loan officers, risk analysts and administrators, runs a legacy core banking system or mainframe, uses credit bureau, KYC and payment gateway APIs, and must follow lending, privacy, KYC/AML and audit rules that differ by country.
2. **Derive the requirements.** The scenario was split into six requirement areas: scale, business rules, integration, security and compliance, tailored screens, and long-term upkeep (Section 3.2).
3. **Draw the integration map.** Every link to the platform was drawn and coloured by how well a typical low-code platform supports it: well supported, possible with effort, or strained (Section 3.1).
4. **Analyse the limits.** Each common limitation was written down with its effect on this bank and a way to reduce it (Section 3.3).
5. **Show the strain.** A conceptual curve and an indicative suitability chart were prepared to show where low-code fits and where it does not (Sections 3.4 and 3.5).
6. **Choose and recommend.** A decision flow was drawn for picking between low-code, hybrid and custom development (Section 3.6), followed by a recommended hybrid architecture (Section 3.7).

Note: this is an analytical case study. Actual limits such as run time, record counts or API calls vary by platform and plan, so no figures are quoted; the documentation of the chosen platform should be checked. The charts are conceptual or indicative, not measured benchmarks.

3. Output

3.1 Enterprise scenario and integration map



The map places a generic low-code platform at the centre of the bank. Built-in platform services and ready-made connectors are well supported (green). Credit bureau, KYC and payment gateway APIs, and the country-by-country lending, privacy and audit rules, are possible with effort (amber). The legacy core banking system or mainframe, and credit scoring or risk analytics, are strained and usually need custom code (red).

3.2 Enterprise requirements

The requirements drawn from the scenario are listed below.

Requirement	Why it is complex
Scale	Applications arrive from many regions, with sharp peaks during festive and promotional seasons.
Business rules	Eligibility, interest, tax and regulatory rules differ by country and change frequently.
System integration	A legacy core banking system or mainframe, credit bureau, KYC and payment APIs, and company-wide sign-in.
Security and compliance	KYC/AML, privacy and financial rules require audit trails, access control and local data storage.
Tailored screens	Applicants, loan officers and risk analysts each need a different interface and language.
Long-term upkeep	Many teams change the system over the years, and the bank may switch platform later.

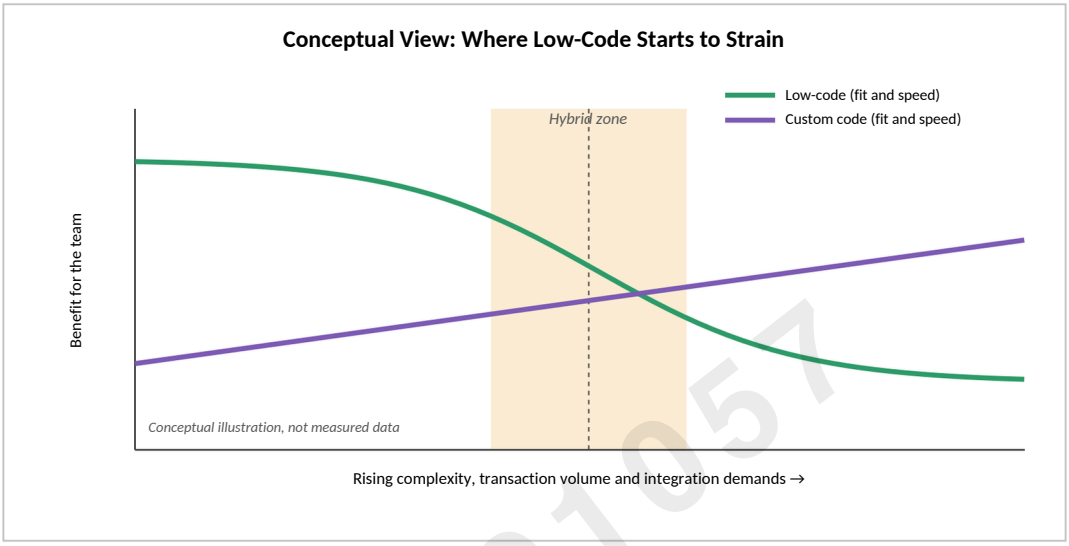
3.3 Limitation analysis

The limitations below are common across low-code platforms. Each is shown with its effect on this bank and a way to reduce it.

Area	Limitation of low-code	Impact on this case	Mitigation
Performance at scale	Shared, vendor-managed runtime with usage caps set by plan.	Peak-season application surges and large reports may slow down or hit caps.	Tune queries, avoid heavy loops, push bulk jobs to external services.
Complex logic	Built-in scripting suits routine rules, not advanced algorithms.	Credit scoring and risk-based pricing are hard to express and maintain.	Place the logic in a custom service and call it by API.
Integration	Ready connectors cover popular tools; legacy and industry-specific interfaces need extra work.	The core banking system, mainframe and bureau systems cannot be joined by a simple connector.	Add an API gateway or middleware between the platform and legacy systems.
Customisation	Interface follows fixed components and platform conventions.	Highly tailored, multi-language screens are difficult to build.	Use custom components or a separate front end over the platform API.
Compliance and data residency	Hosting regions, encryption choices and audit depth depend on the vendor.	Some countries may require more control than the platform gives.	Check vendor certifications, restrict access by role, store sensitive data in a controlled system.
Testing and change control	Version history, automated tests and release tools are often basic.	A faulty eligibility or interest rule change could reach live applicants unnoticed.	Use test environments, peer review and a written release routine.
Sprawl	Easy building lets each team create its own apps.	Duplicate, insecure or unsupported loan apps appear across regions.	Keep a central app register, shared standards and a named owner per app.
Vendor lock-in	Apps and workflows are bound to one platform and hard to export.	A later change of platform means rebuilding from scratch.	Keep core logic in APIs and keep data exportable.

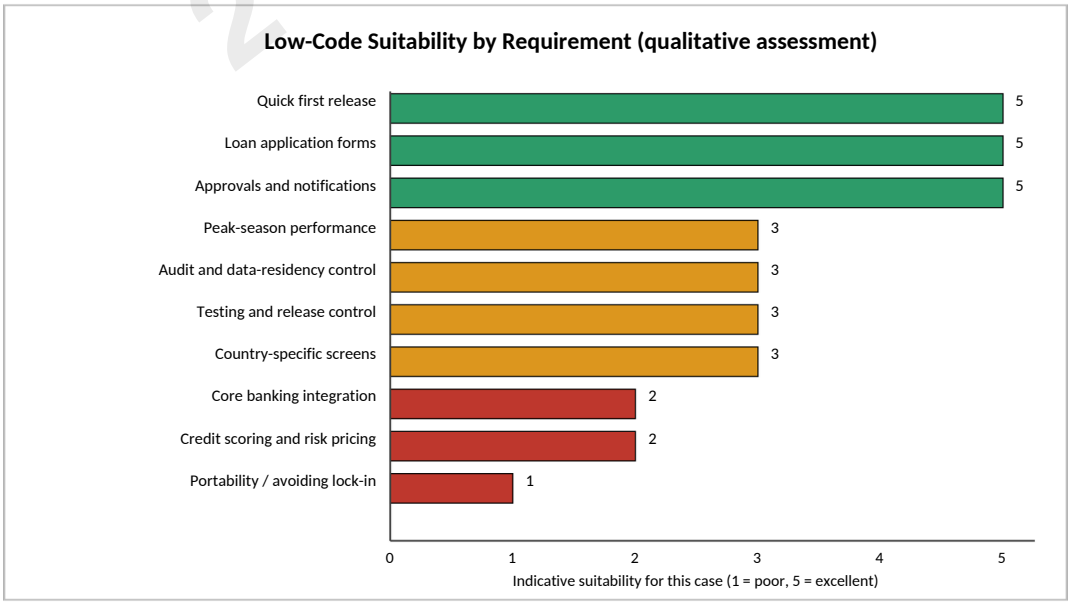
24140457

3.4 Where low-code starts to strain



The curve shows a large benefit from low-code for simple needs. As complexity, transaction volume and integration demands increase, that benefit falls, while custom code stays fairly steady. The shaded hybrid zone marks where using both together is usually best. The chart is a conceptual illustration, not measured data.

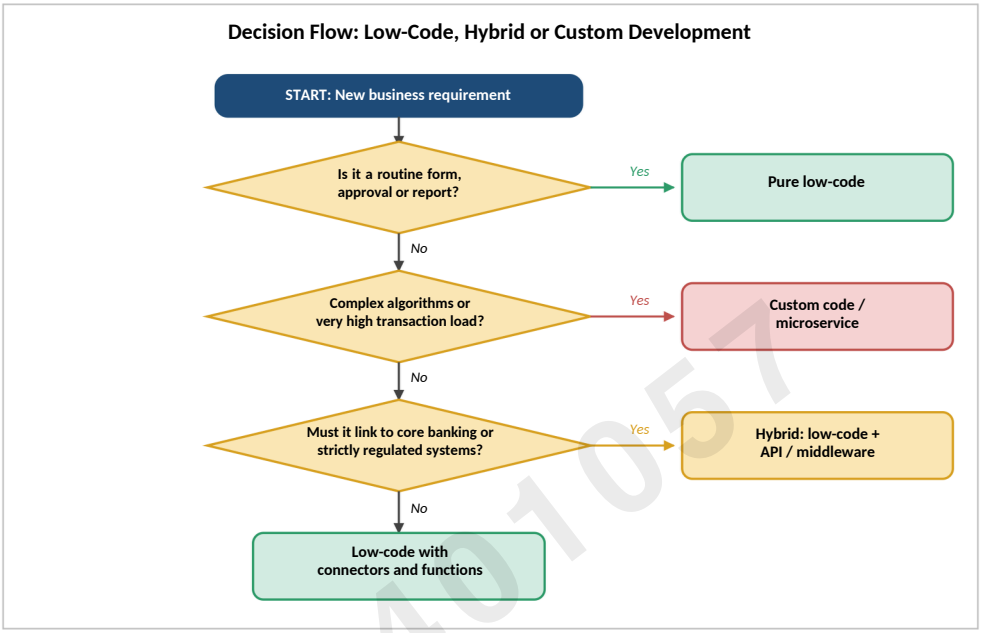
3.5 Suitability of low-code by requirement



The chart rates how well low-code fits each requirement in this case. A quick first release, loan application forms, and approvals score highest. Credit scoring, core banking integration and portability score lowest. The values are indicative scores from this analysis, not benchmark results.

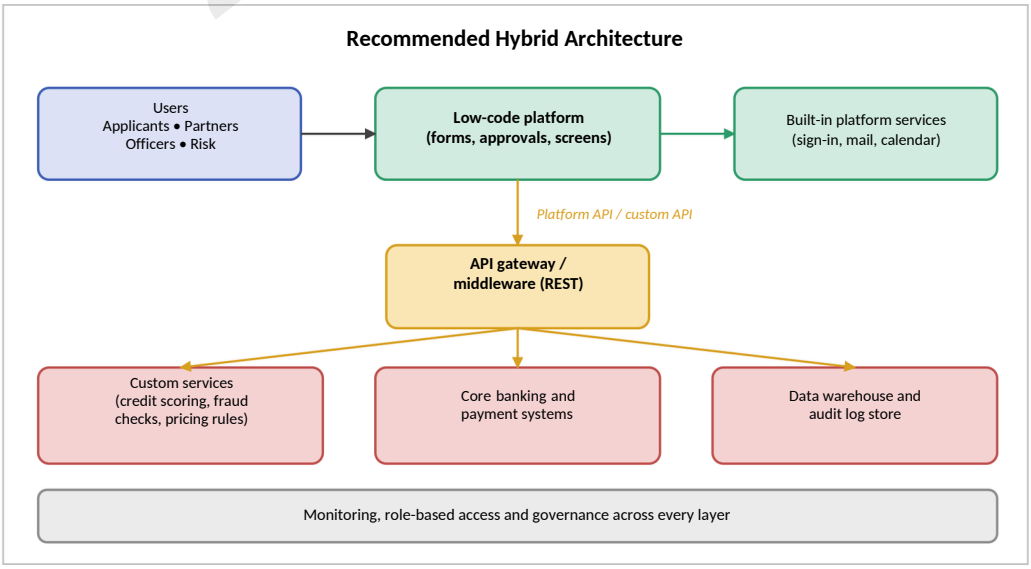
24110157

3.6 Decision flow for choosing the approach



The flow asks three questions: is the need a routine form, approval or report; does it require complex algorithms or very high transaction load; and must it link to core banking or strictly regulated systems. The answers lead to pure low-code, custom code, a hybrid design, or low-code with connectors and functions.

3.7 Recommended hybrid architecture



The low-code platform keeps the forms, approvals and screens, and relies on built-in services for sign-in and messages. An API gateway or middleware layer links it to custom services for credit scoring and fraud checks, to the core banking and payment systems, and to a data warehouse and audit store. Monitoring, role-based access and governance cover every layer.

24140157

4. Final Result

The case study found that low-code is very effective for the routine parts of an enterprise lending system: loan application forms, approvals, notifications and ecosystem integrations. It delivers quickly with little code, whichever platform is chosen.

For a complex, global use case, the main limitations were performance at scale, complex logic, legacy and industry integration, restricted customisation, compliance and data-residency control, weak testing and change control, app sprawl and vendor lock-in. These were examined in the limitation table, the conceptual chart and the suitability chart.

The recommended solution is a hybrid approach: keep low-code for forms, workflows and screens, and add an API or middleware layer with custom services for heavy logic, core banking integration and analytics, under clear governance. No live testing or benchmark was carried out; the conclusions come from analysis of the case.

241401057